

PROJECT 5 REPORT

Junrui Ding, Junyao Han

Overall Project Description

This project explores building, training, analyzing, and modifying deep neural networks for image recognition. Using the MNIST handwritten digit dataset, we built a convolutional neural network (CNN) from scratch, examined its learned filters, applied transfer learning to recognize Greek letters, re-implemented the network using transformer layers, and conducted an ablation study to evaluate how different architectural choices affect performance.

Task 1: Build and Train a Network to Recognize Digits

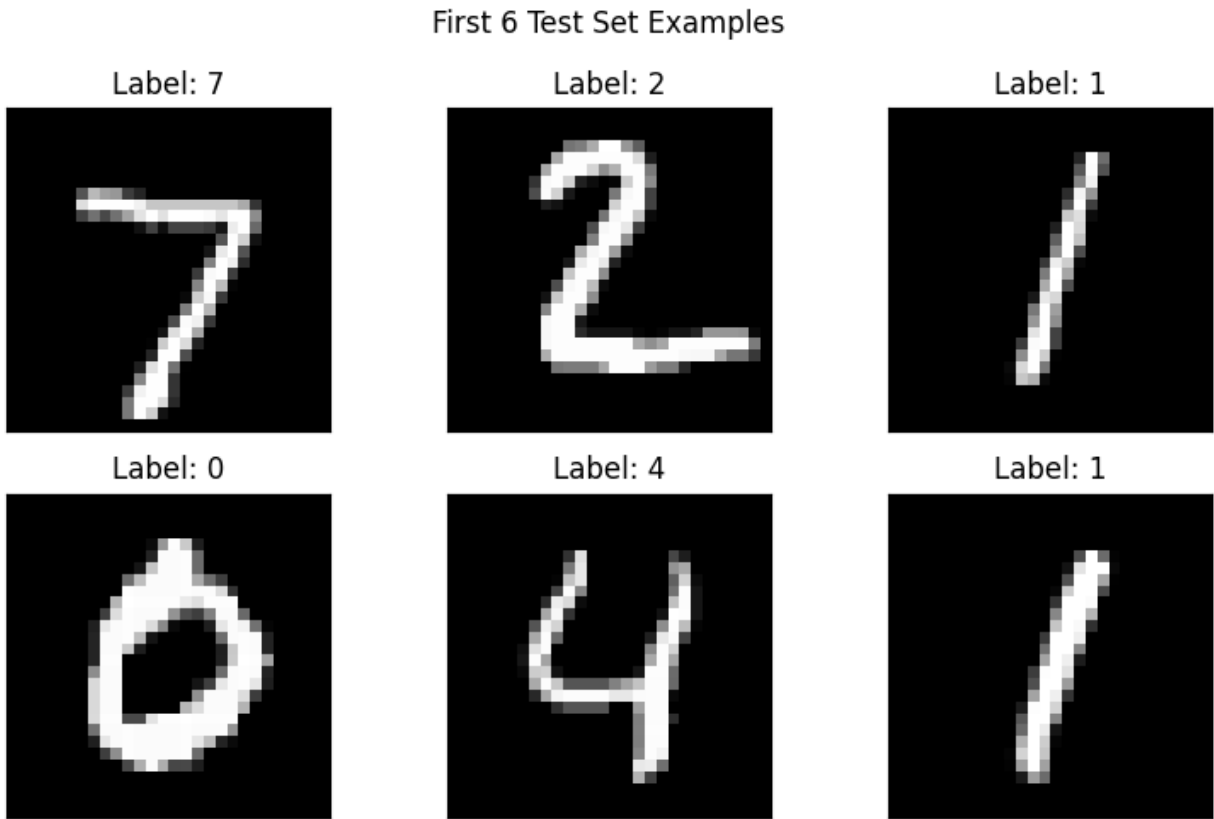
For this task, we built a CNN using PyTorch and trained it on the MNIST digit dataset (60,000 training images, 10,000 test images, each 28×28 grayscale).

The network architecture consists of:

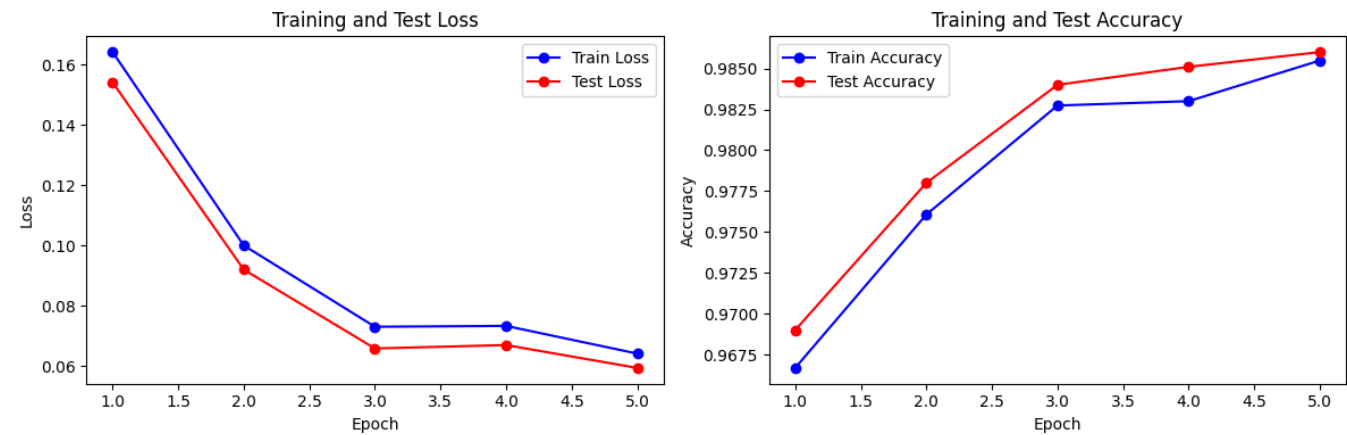
- A convolutional layer with 10 5×5 filters
- A max pooling layer (2×2) with ReLU activation
- A second convolutional layer with 20 5×5 filters
- A dropout layer (rate=0.5)
- A second max pooling layer with ReLU
- A fully connected layer with 50 nodes and ReLU
- A final fully connected layer with 10 nodes and log-softmax output

The model was trained using SGD with momentum=0.5, learning rate=0.01, and batch size=64 for 5 epochs.

The first 6 examples from the test set are shown below:



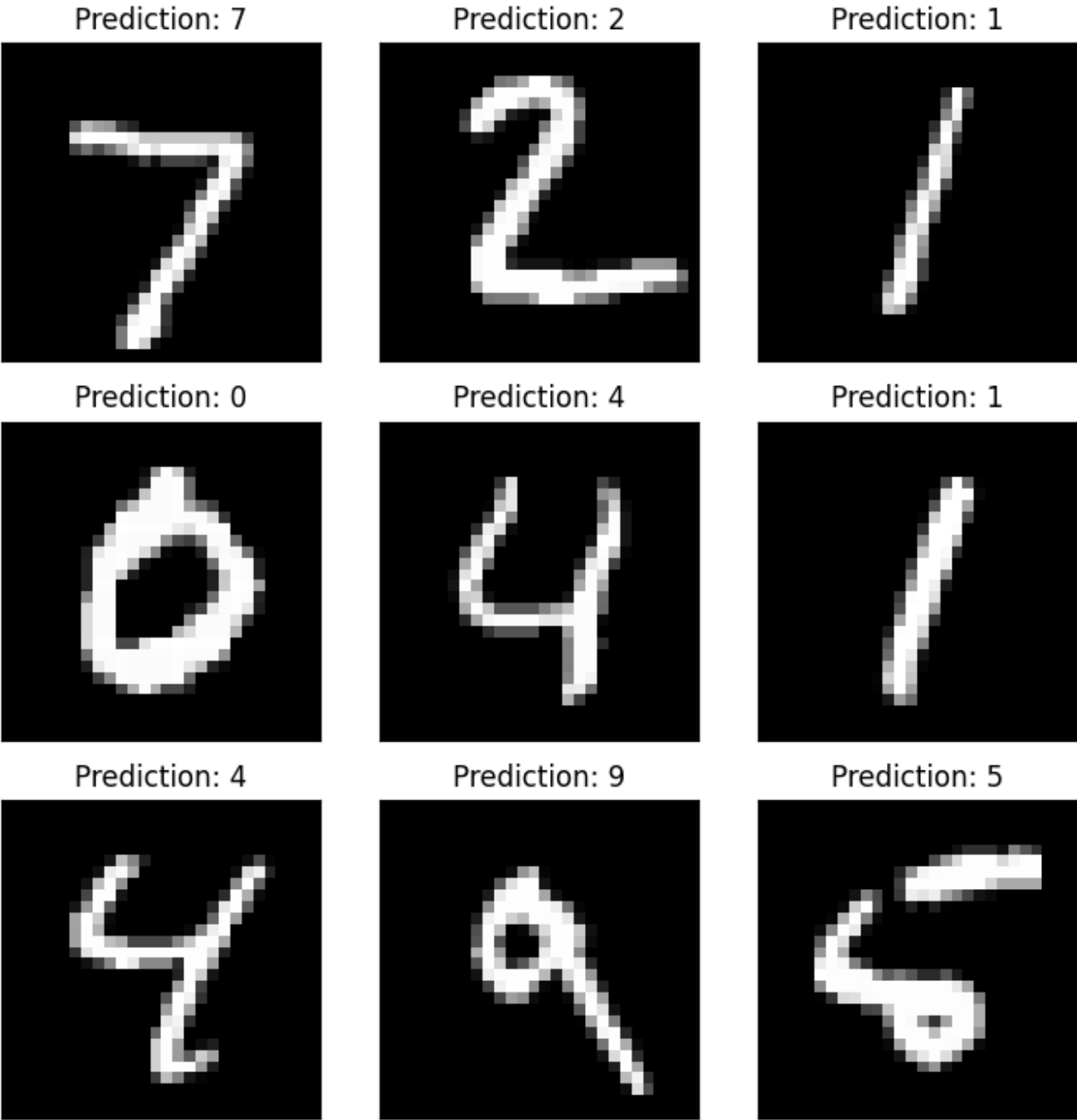
As shown in the figure below, both training and test loss decreased steadily across epochs, while accuracy improved from ~96% to ~98.6% on the test set. The test accuracy consistently matched or slightly exceeded training accuracy, indicating the model generalized well without overfitting.



We then ran the trained model on the first 10 examples of the test set. The model correctly classified all 10 examples. The output log-probabilities, predicted labels, and correct labels are shown in the table below:

```
(venv) jrd@JunruideMacBook-Pro src % python mnist_test.py
Index  Output Values                                Predicted  Correct
-----
0      ['-10.18', '-10.94', '-6.68', '-8.12', '-13.43', '-12.91', '-18.77', '-0.00', '-10.43', '-9.47'] 7      7
1      ['-6.67', '-5.40', '-0.01', '-9.30', '-13.14', '-14.59', '-9.48', '-12.03', '-7.66', '-15.85'] 2      2
2      ['-6.35', '-0.03', '-5.75', '-8.35', '-4.56', '-7.32', '-6.24', '-4.83', '-6.85', '-8.85'] 1      1
3      ['-0.00', '-12.48', '-7.34', '-9.71', '-13.07', '-10.16', '-7.82', '-8.49', '-10.20', '-7.93'] 0      0
4      ['-9.35', '-11.61', '-9.77', '-14.03', '-0.00', '-11.32', '-10.56', '-8.61', '-11.81', '-6.08'] 4      4
5      ['-7.61', '-0.01', '-7.50', '-10.26', '-5.33', '-9.19', '-8.24', '-5.85', '-8.33', '-10.24'] 1      1
6      ['-13.76', '-7.10', '-10.18', '-10.94', '-0.02', '-7.05', '-11.08', '-5.73', '-5.14', '-5.29'] 4      4
7      ['-9.55', '-7.14', '-4.72', '-4.33', '-2.97', '-5.17', '-11.28', '-4.74', '-4.52', '-0.11'] 9      9
8      ['-9.32', '-10.34', '-7.48', '-9.53', '-7.64', '-0.03', '-3.60', '-9.17', '-5.63', '-6.33'] 5      5
9      ['-9.34', '-12.69', '-9.73', '-6.90', '-5.73', '-9.54', '-13.71', '-2.58', '-5.01', '-0.09'] 9      9
0.png: Predicted = 0
1.png: Predicted = 6
2.png: Predicted = 2
3.png: Predicted = 2
4.png: Predicted = 8
5.png: Predicted = 5
6.png: Predicted = 1
7.png: Predicted = 7
8.png: Predicted = 8
9.png: Predicted = 1
```

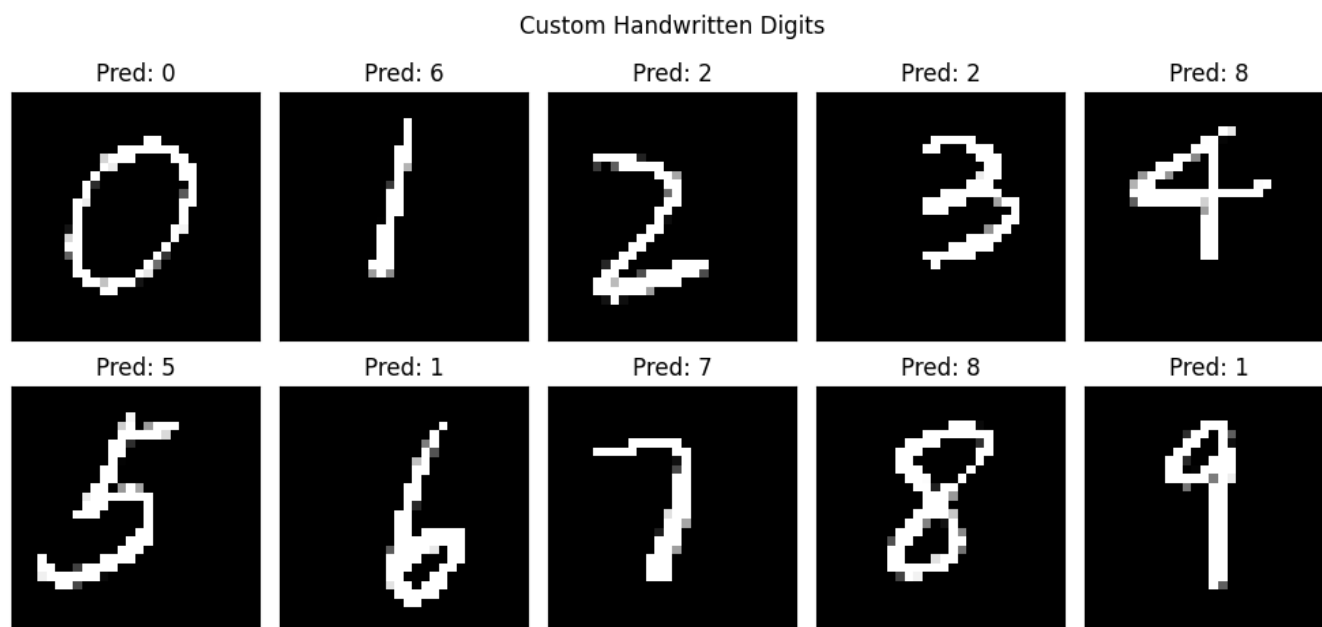
The first 9 test set predictions are shown below in a 3x3 grid:



Task 1F: Testing on Custom Handwritten Digits

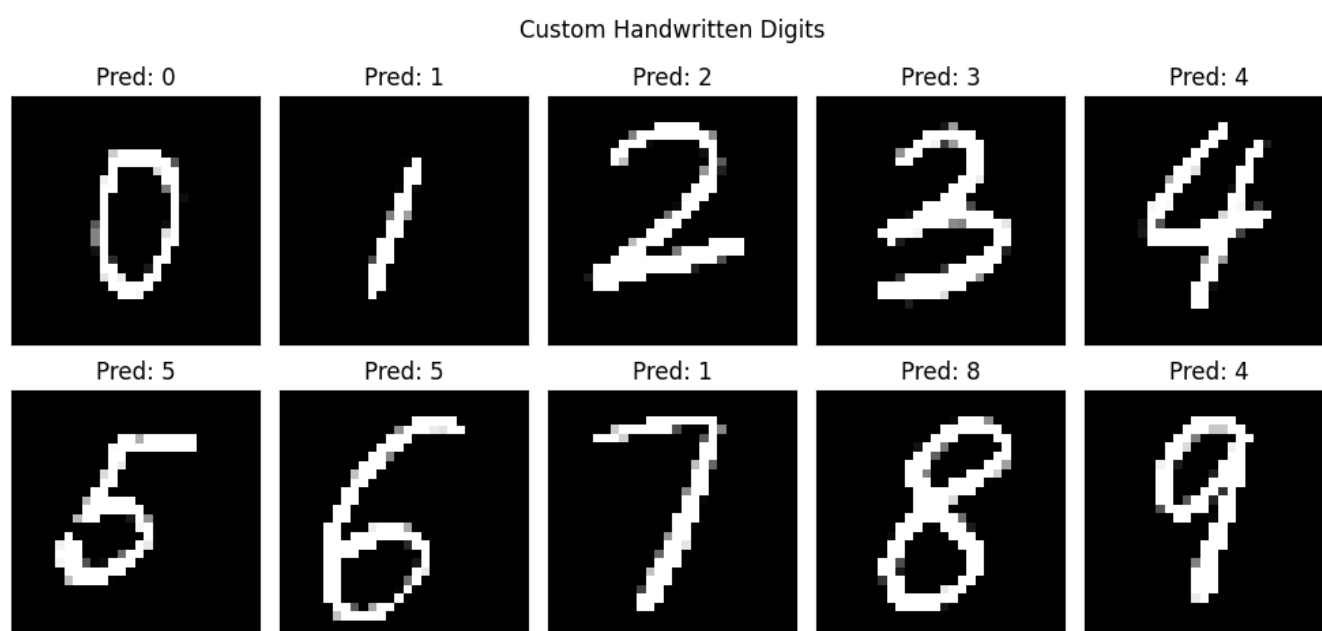
We tested the network on handwritten digits [0–9] written on a tablet using a stylus in Notability. Images were individually cropped, and the preprocessing pipeline automatically inverted intensities to match MNIST format (white digit on black background).

Version 1 (original handwriting style) — 4/10 accuracy:



Errors: 1→6, 4→8, 6→1, 9→1.

Version 2 (revised to closer match MNIST style) — 6/10 accuracy:



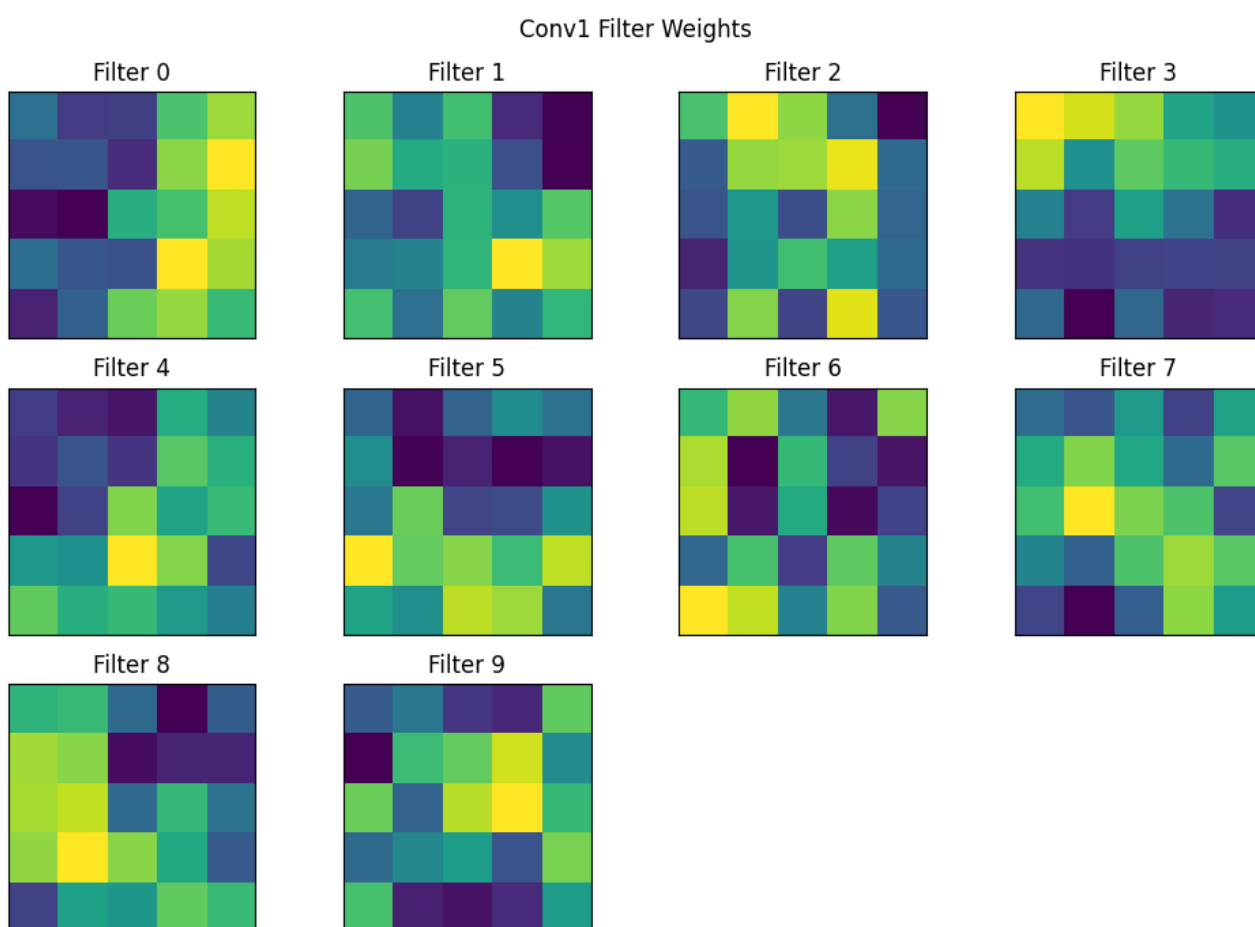
Errors: 6→5, 7→1, 9→4.

Analysis of the preprocessed images confirmed correct formatting (white digit on black background, centered), so errors were attributed to handwriting style differences from the MNIST training distribution. In

v1, a heavily slanted 1, closed-top 4, and stylized 6 and 9 caused misclassifications. In v2, revising to straighter strokes and an open-top 4 improved accuracy from 4/10 to 6/10. The remaining misclassified 7 (predicted as 1) likely resulted from a thin horizontal stroke at the top — without a prominent horizontal feature, the dominant visual pattern becomes a single diagonal line, which closely resembles a 1 in the MNIST distribution. The errors suggest the model is sensitive to stroke weight and style variations not well-represented in the training data.

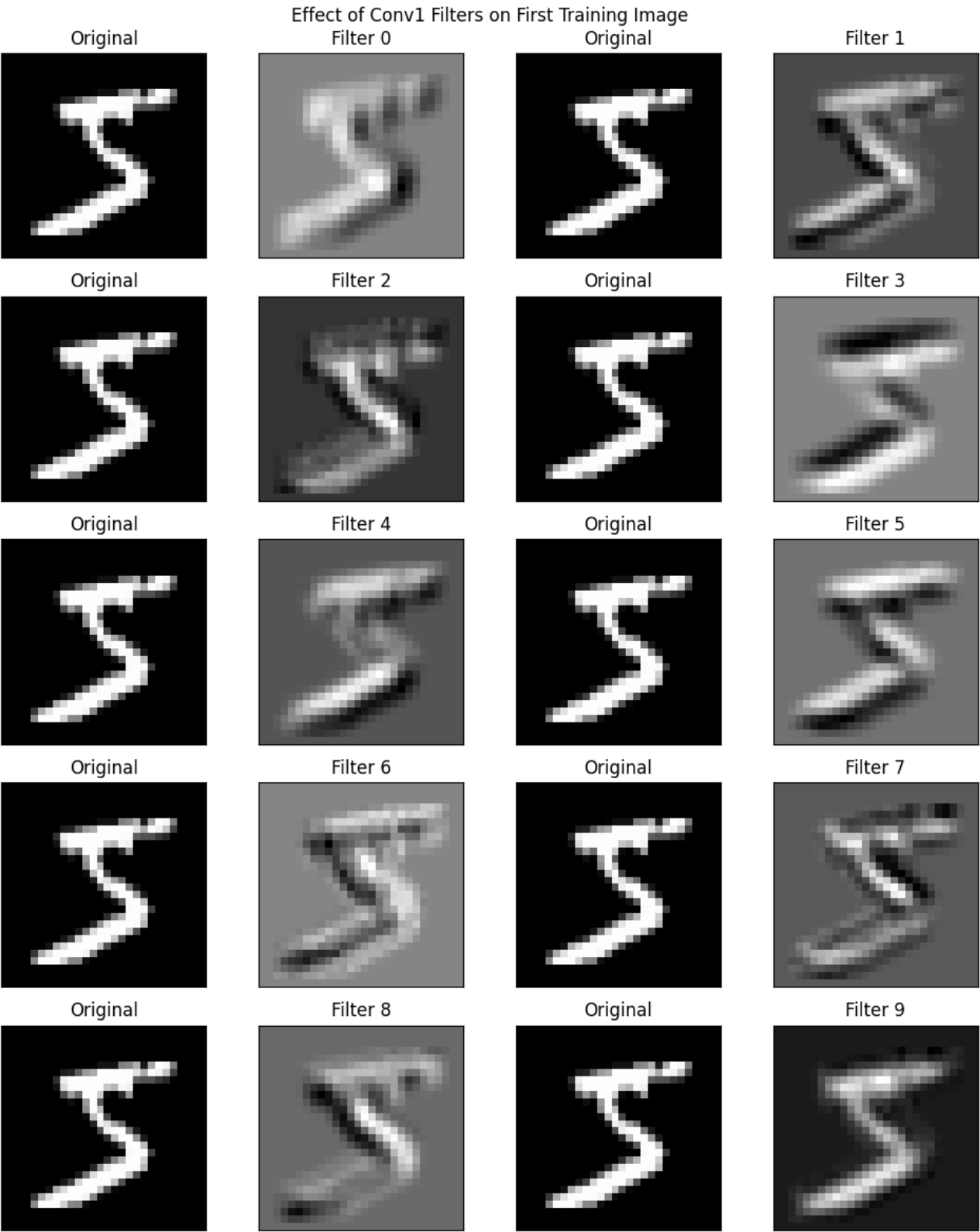
Task 2: Examine the Network

We loaded the trained model and printed its structure to identify layer names. We then extracted the weights of the first convolutional layer (`conv1`), which has shape `[10, 1, 5, 5]` — 10 filters, each 5×5.



The filters show varied spatial patterns with no two filters identical, indicating the network has learned to detect different local features. Some filters (e.g., Filter 0, 7) have strong localized bright spots suggesting they respond to specific corners or blob-like structures. Others (e.g., Filter 3, 6) show more gradient-like patterns across the spatial extent, likely responding to oriented edges or strokes. This diversity is expected and desirable — each filter specializes in a different aspect of the input.

We then applied all 10 filters to the first training image (digit "5") using OpenCV's `filter2D` function:

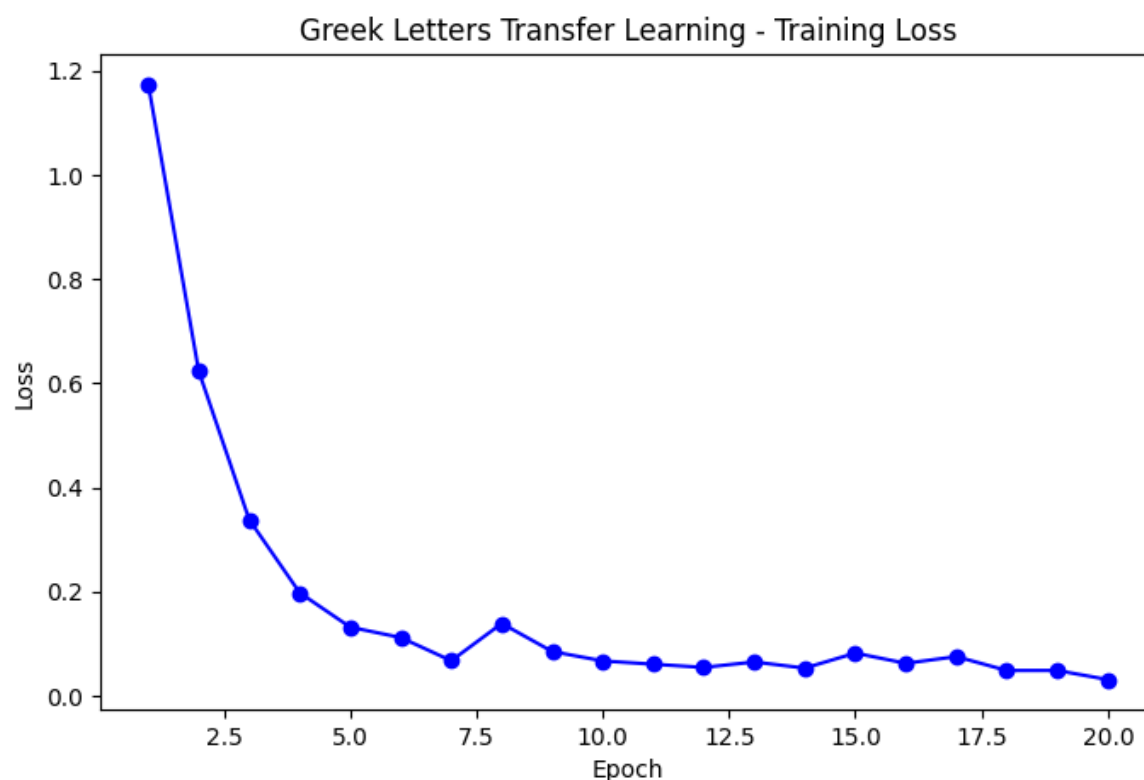


The results make sense given the filter shapes. Some filters produce smoothing/brightening effects (e.g., Filter 0), preserving the overall stroke structure. Others produce strong edge enhancement (e.g., Filter 2, 6), highlighting the boundaries and directional strokes of the digit. Filters with opposing positive/negative weights (e.g., Filter 3) produce a gray background with sharp contrast at stroke edges, consistent with edge detection behavior.

Task 3: Transfer Learning on Greek Letters

For this task, we adapted the pretrained MNIST CNN to classify three Greek letters: alpha, beta, and gamma, using only 27 training examples (9 per class). We froze all network weights except the final fully connected layer, which was replaced with a new 3-node output layer. Only this new layer (153 parameters) was trained — the rest of the network retained its MNIST-learned features.

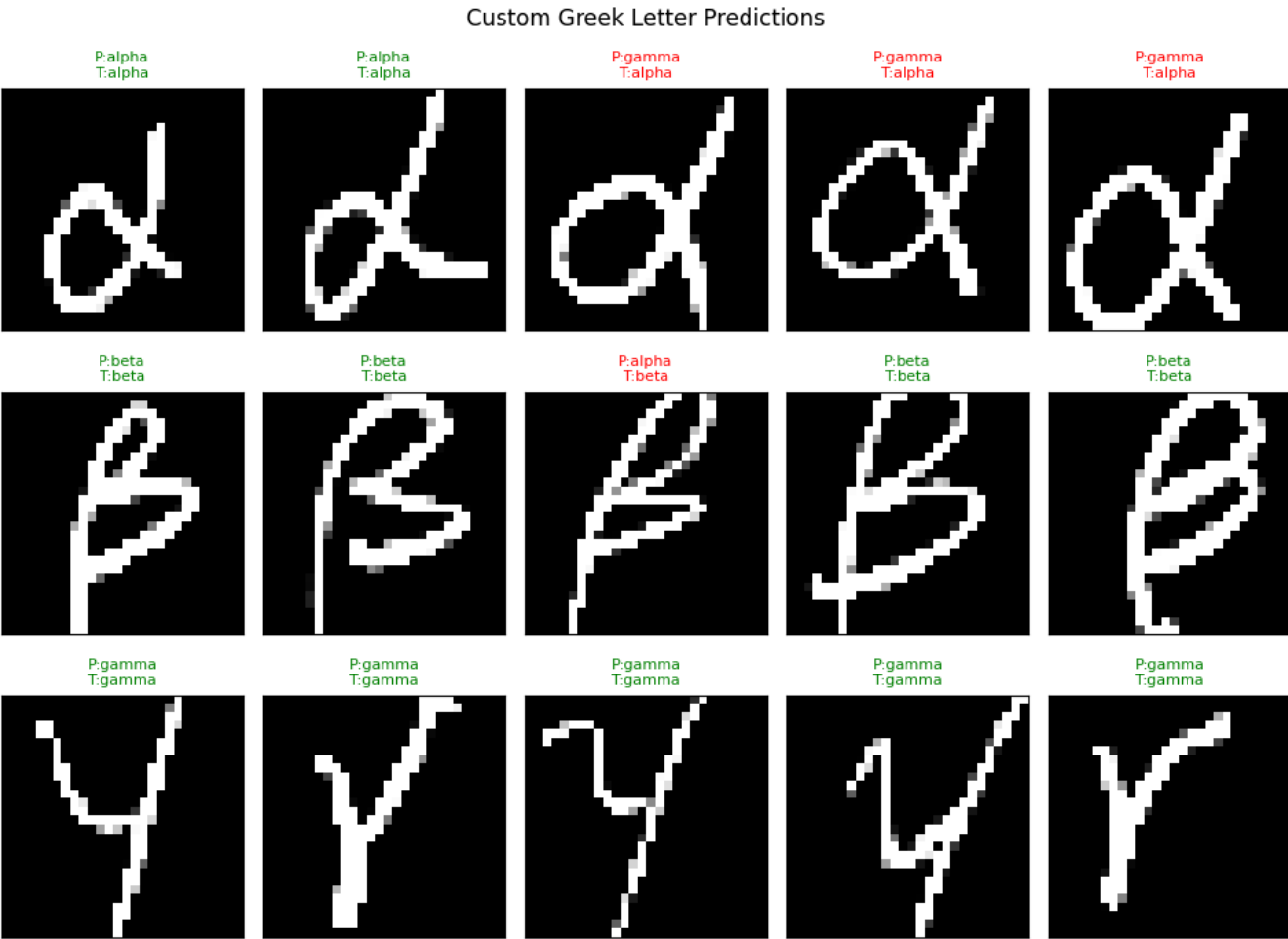
The modified network structure is identical to the original except for the final layer: `fc2: Linear(in_features=50, out_features=3)`.



As shown in the figure, the loss dropped sharply in the first 5 epochs and continued to decrease gradually, approaching near-zero by epoch 20. The network reached near-perfect classification on the training set within approximately 5 epochs. This demonstrates the power of transfer learning — the convolutional features learned on MNIST digits (edges, strokes, curves) transferred effectively to the Greek letter recognition task, even with only 27 examples.

The network reached 100% accuracy (27/27) on the training set by epoch 9 and maintained it through epoch 20. The rapid convergence with only 27 examples confirms that the frozen convolutional features were already well-suited for this task.

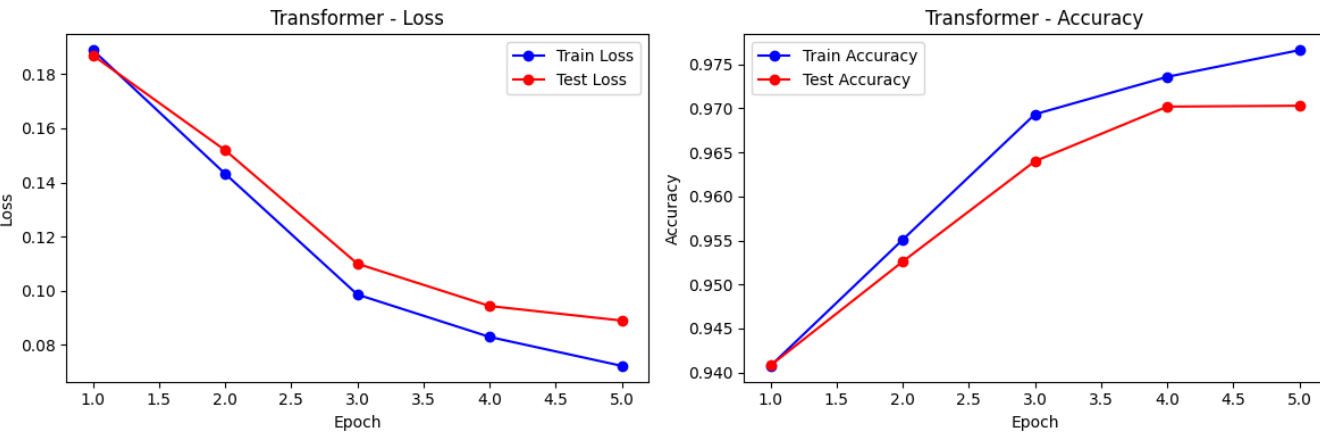
We also tested the model on our own handwritten alpha, beta, and gamma symbols (5 images per class, 15 total). The model achieved **13/15 accuracy (87%)**. The two misclassified examples were alpha images predicted as gamma, likely due to writing style variations in the loop shape of alpha. All beta and gamma examples were correctly classified.



Green titles indicate correct predictions; red indicates misclassification.

Task 4: Re-implement with Transformer Layers

For this task, we replaced the CNN with a Vision Transformer (ViT)-style model. The 28×28 image is divided into 16 non-overlapping 7×7 patches. Each patch is linearly embedded into a 64-dimensional token, positional embeddings are added, and the sequence is processed by 2 transformer encoder layers (4 attention heads, MLP dimension 128). The output tokens are averaged to produce a single representation, which is passed through a classification head (Linear → ReLU → Linear with 10 outputs).



The transformer model achieved **97.1% test accuracy** after 5 epochs, compared to **98.6%** for the CNN. The transformer trained noticeably slower on CPU due to the attention computation overhead. The gap in accuracy is expected — transformers generally require more data and longer training to match CNNs on small datasets like MNIST, where local spatial structure (well-captured by convolutions) is the dominant feature.

Task 5: Design Your Own Experiment

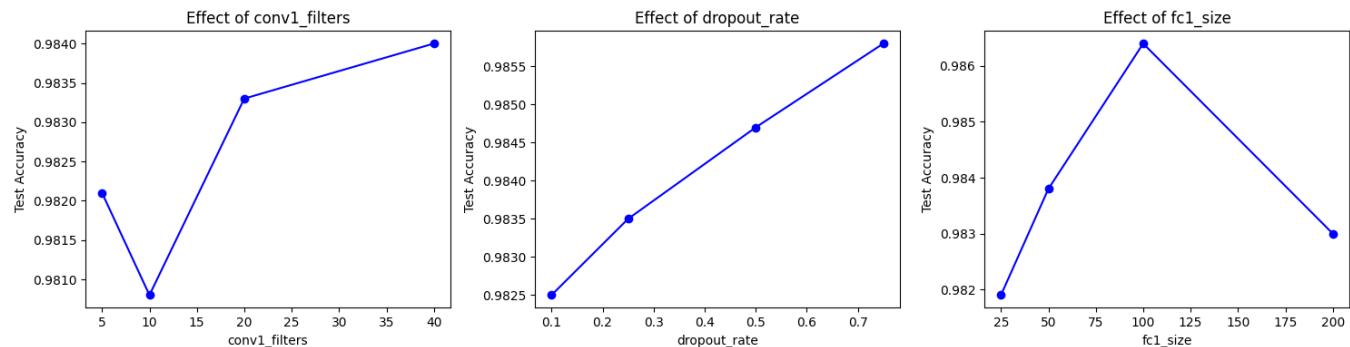
We conducted an ablation study on the CNN architecture, varying three dimensions one at a time using a linear search strategy. Each configuration was trained for 3 epochs and evaluated on the test set. A total of 12 network variants were evaluated.

Dimensions explored:

- 1. Number of conv1 filters: [5, 10, 20, 40]
- 2. Dropout rate: [0.1, 0.25, 0.5, 0.75]
- 3. FC1 hidden size: [25, 50, 100, 200]

Hypotheses (before running):

- More conv1 filters → higher accuracy up to a point, then diminishing returns
- Higher dropout → lower accuracy (too much regularization hurts with this dataset size)
- Larger FC1 → marginal improvement (MNIST bottleneck is in the conv layers)



Results:

Dimension	Value	Test Accuracy
conv1_filters	5	98.21%
conv1_filters	10	98.08%
conv1_filters	20	98.33%
conv1_filters	40	98.40%
dropout_rate	0.10	98.25%
dropout_rate	0.25	98.35%
dropout_rate	0.50	98.47%

Dimension	Value	Test Accuracy
dropout_rate	0.75	98.58%
fc1_size	25	98.19%
fc1_size	50	98.38%
fc1_size	100	98.64%
fc1_size	200	98.30%

Discussion:

The results partially supported our hypotheses. For conv1 filters, accuracy generally increased with more filters (5→40: 98.21%→98.40%), though the differences were small — consistent with diminishing returns on a simple dataset like MNIST. Notably, 10 filters performed slightly worse than 5, suggesting some stochasticity in 3-epoch training.

For dropout rate, the result was surprising and contradicted our hypothesis: higher dropout consistently improved accuracy, with 0.75 achieving the best result (98.58%). This suggests the baseline model was slightly underfitting rather than overfitting, and stronger regularization helped generalization.

For FC1 size, accuracy peaked at 100 nodes (98.64%) and dropped at 200, matching our hypothesis of an optimal middle ground — too small underfits, too large may overfit or add noise with only 3 epochs of training.

Reflection

Junrui: This project helped me understand how convolutional neural networks learn hierarchical features from images, and how transfer learning allows a pretrained model to be repurposed for a new task with very little data. Seeing the learned filters and their effects on actual images made the abstract concept of feature detection much more concrete. I was also surprised by how quickly the transformer model trained on CPU — though it achieved slightly lower accuracy than the CNN, it demonstrated that attention-based architectures can be competitive even on simple datasets.

Junyao:

Acknowledgement

This project was completed collectively by Junrui Ding and Junyao Han. We referenced the PyTorch documentation and tutorials for network architecture and training loop structure. The Greek letter dataset and **GreekTransform** code were provided by the course instructor. We also used AI tools (Claude Code) for debugging help, implementation guidance, code explanation, and report drafting support.